computed using the formula `firstBackoff * (factor ^ n)`. If you set the `basedOnPreviousValue` parameter to `true`, the formula will be `prevBackoff * factor`.

Listing 9.5 Applying the retry filter to all routes

```
spring:
  cloud:
    gateway:
      default-filters:                                    ❶
        - name: Retry                                     ❷
          args:
            retries: 3                                    ❸
            methods: GET                                  ❹
            series: SERVER_ERROR                          ❺
            exceptions: java.io.IOException,
            ➥ java.util.concurrent.TimeoutException      ❻
            backoff:                                      ❼
              firstBackoff: 50ms
              maxBackOff: 500ms
              factor: 2
              basedOnPreviousValue: false
```

❶ A list of default filters

❷ The name of the filter

❸ Maximum of 3 retry attempts

❹ Retries only GET requests

❺ Retries only when 5XX errors

❻ Retries only when the given exceptions are thrown

❼ Retries with a delay computed as “firstBackoff * (factor ^ n)”

The retry pattern is useful when a downstream service is momentarily unavailable. But what if it stays down for more than a few instants? At that point we could stop forwarding requests to it until we’re sure that it’s back. Continuing to send requests won’t be beneficial for the caller or the callee. In that scenario, the circuit breaker pattern comes in handy. That’s the topic of the next section.